```
import numpy as np
import pandas as pd
from numpy import log
```

```
def squared_error(y, yhat):
    return (y - yhat) ** 2

squared_error(1, 1)
```

```
0
```

```
base_cases = [
    (1, 1),
    (1, 0.9997),
    (1, 0.9),
    (1, 0.6),
    (1, 0.1192),
    (0, 0.1192),
]

rows = []
for y, yhat in base_cases:
    rows.append({
        "y": y,
        "yhat": yhat,
        "squared_error": squared_error(y, yhat),
    })

df_q = pd.DataFrame(rows)
df_q_rounded = df_q.copy()
df_q_rounded["squared_error"] = df_q_rounded["squared_error"].round(6)
df_q_rounded
```

	y	yhat	squared_error
0	1	1.0000	0.000000
1	1	0.9997	0.000000
2	1	0.9000	0.010000
3	1	0.6000	0.160000
4	1	0.1192	0.775809
5	0	0.1192	0.014209

Next steps: Generate code with df_q_rounded | New interactive sheet

```
extra_cases = [
    (0, 0.0),
    (0, 0.5),
    (0, 0.9),
    (1, 0.2),
    (1, 0.05),
]

rows = []
for y, yhat in extra_cases:
    rows.append({
        "y": y,
        "yhat": yhat,
        "squared_error": squared_error(y, yhat),
    })

df_q_extra = pd.DataFrame(rows)
df_q_extra
```

	y	yhat	squared_error
0	0	0.00	0.0000
1	0	0.50	0.2500
2	0	0.90	0.8100
3	1	0.20	0.6400
4	1	0.05	0.9025

Next steps: Generate code with df_q_extra | New interactive sheet

```
def mean_quadratic_cost(y, yhat):
    y = np.asarray(y, dtype=float)
    yhat = np.asarray(yhat, dtype=float)
    return np.mean((y - yhat) ** 2)

mean_quadratic_cost(df_q["y"], df_q["yhat"])
```

```
np.float64(0.160002895)
```

```
def cross_entropy(y, a, eps=1e-15):
    a = np.clip(a, eps, 1 - eps)
    return -(y * log(a) + (1 - y) * log(1 - a))

cross_entropy(1, 0.9)
```

```
np.float64(0.10536051565782628)
```

```
base_cases_ce = [
    (1, 0.9997),
    (1, 0.9),
    (1, 0.6),
    (1, 0.1192),
    (0, 0.1192),
    (1, 1 - 0.1192),
]

rows = []
for y, a in base_cases_ce:
    rows.append({
        "y": y,
        "a": a,
        "cross_entropy": cross_entropy(y, a),
    })

df_ce = pd.DataFrame(rows)
df_ce_rounded = df_ce.copy()
df_ce_rounded["cross_entropy"] = df_ce_rounded["cross_entropy"].round(4)
df_ce_rounded
```

	y	a	cross_entropy
0	1	0.9997	0.0003
1	1	0.9000	0.1054
2	1	0.6000	0.5108
3	1	0.1192	2.1270
4	0	0.1192	0.1269
5	1	0.8808	0.1269

Next steps: Generate code with df_ce_rounded | New interactive sheet

```
extra_cases_ce = [
    (1, 0.99),
    (1, 0.01),
    (1, 0.5),
    (0, 0.01),
    (0, 0.99),
]

rows = []
for y, a in extra_cases_ce:
```

```
    rows.append({
        "y": y,
        "a": a,
        "cross_entropy": cross_entropy(y, a),
    })

df_ce_extra = pd.DataFrame(rows)
df_ce_extra["cross_entropy"] = df_ce_extra["cross_entropy"].round(4)
df_ce_extra
```

	y	a	cross_entropy
0	1	0.99	0.0101
1	1	0.01	4.6052
2	1	0.50	0.6931
3	0	0.01	0.0101
4	0	0.99	4.6052

Next steps: Generate code with `df_ce_extra` New interactive sheet

```
def mean_cross_entropy(y, a, eps=1e-15):
    y = np.asarray(y, dtype=float)
    a = np.asarray(a, dtype=float)
    a = np.clip(a, eps, 1 - eps)
    return np.mean(-(y * log(a) + (1 - y) * log(1 - a)))

mean_cross_entropy(df_ce["y"], df_ce["a"])
```

```
np.float64(0.4995480159885067)
```